

Zadania

- 1.1 Pod adresem `http://127.0.0.1:10001` działa prosty serwer HTTP udostępniający dwa endpointy: `http://127.0.0.1:10001/hash` oraz `http://127.0.0.1:10001/submit`.

Twoim zadaniem jest obliczenie skrótu MD5 podanego przez serwer losowego słowa. Wynikowy hash powinien być w formacie szesnastkowym (hex). Aby otrzymać od serwera słowo do zahashowania oraz identyfikator sesji, wyślij zapytanie HTTP GET do endpointa `http://127.0.0.1:10001/hash`. Otrzymasz w odpowiedzi unikalny identyfikator sesji oraz losowe słowo do zahashowania.

Następnie powinieneś obliczyć skrót MD5 otrzymanego słowa, kodując wynik w formacie szesnastkowym (hex).

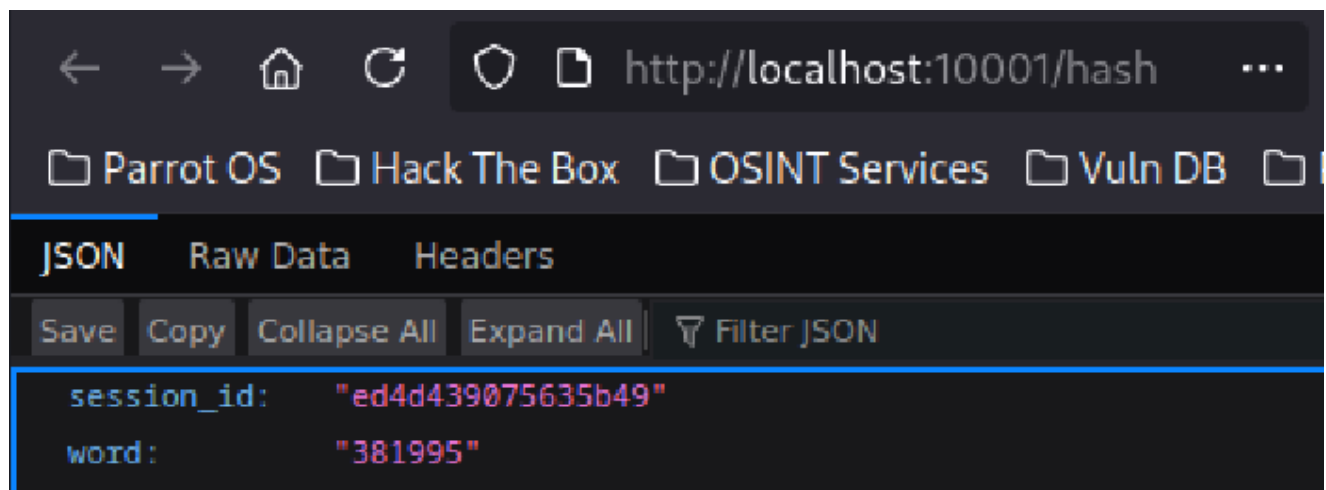
Po obliczeniu hash'a wyślij go do serwera metodą HTTP POST na endpoint `http://127.0.0.1:10001/submit`, przekazując w formacie JSON zarówno identyfikator sesji, jak i obliczony skrót MD5 (hex). Serwer zweryfikuje poprawność przesłanego hasha i zwróci informację o sukcesie lub błędzie.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 10001:10001 mazurkatarzyna/hashing-md5-ex1:latest
```

- (b) Do obliczenia skrótu MD5 użyj narzędzia OpenSSL.

- (c) Aby wysłać odpowiedź do serwera, użyj narzędzia cURL. Pamiętaj o dołączeniu nagłówka HTTP "Content-Type: application/json".



```
[user@parrot]~$ echo -n "381995" | openssl dgst -md5
MD5(stdin)= 8bb22720613e3484a852a30f0f7c5b1a
[user@parrot]~$ curl -X 'POST' \
'http://localhost:10001/submit' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "ed4d439075635b49",
  "hash_hex": "8bb22720613e3484a852a30f0f7c5b1a"
}'
{"result": "Correct MD5 hash!"}
```

1.2 Pod adresem `http://127.0.0.1:10002` działa prosty serwer HTTP udostępniający dwa endpointy: `http://127.0.0.1:10002/hash` oraz `http://127.0.0.1:10002/submit`.

Twoim zadaniem jest obliczenie skrótu SHA-256 podanego przez serwer losowego słowa. Wynikowy hash powinien być w formacie szesnastkowym (hex). Aby otrzymać od serwera słowo do zahashowania oraz identyfikator sesji, wyślij zapytanie HTTP GET do endpointa `http://127.0.0.1:10002/hash`. Otrzymasz w odpowiedzi unikalny identyfikator sesji oraz losowe słowo do zahashowania.

Następnie powinieneś obliczyć skrót SHA-256 otrzymanego słowa, kodując wynik w formacie szesnastkowym (hex).

Po obliczeniu hash'a wyślij go do serwera metodą HTTP POST na endpoint `http://127.0.0.1:10002/submit`, przekazując w formacie JSON zarówno identyfikator sesji, jak i obliczony skrót MD5 (hex). Serwer zweryfikuje poprawność przesłanego hasha i zwróci informację o sukcesie lub błędzie.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 10002:10002 mazurkatarzyna/hashing-sha256-ex1:latest
```

(b) Do obliczenia skrótu SHA-256 użyj narzędzia OpenSSL.

(c) Aby wysłać odpowiedź do serwera, użyj narzędzia cURL. Pamiętaj o dołączeniu nagłówka HTTP "Content-Type: application/json".

```

[user@parrot]~$ curl -X GET http://localhost:10002/hash
{"session_id":"c59c76a04ef91a3d","word":"hustla6"}
[user@parrot]~$ echo -n "hustla6" | openssl dgst -SHA-256
SHA2-256(stdin)= 1ab50c034f5f2972e55080e3bbb9ba805585e8ae9dc0bc9b6a4c07006d534619
[user@parrot]~$ curl -X 'POST' \
'http://localhost:10002/submit' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "c59c76a04ef91a3d",
  "hash_hex": "1ab50c034f5f2972e55080e3bbb9ba805585e8ae9dc0bc9b6a4c07006d534619"
}'
{"result":"Correct SHA-256 hash!"}

```

1.3 Pod adresem `http://127.0.0.1:10003` działa prosty serwer HTTP udostępniający dwa endpointy: `http://127.0.0.1:10003/hash` oraz `http://127.0.0.1:10003/submit`.

Twoim zadaniem jest obliczenie skrótu SHA-512 podanego przez serwer losowego słowa. Wynikowy hash powinien być w formacie szesnastkowym (hex). Aby otrzymać od serwera słowo do zahashowania oraz identyfikator sesji, wyślij zapytanie HTTP `GET` do endpointa `http://127.0.0.1:10003/hash`. Otrzymasz w odpowiedzi unikalny identyfikator sesji oraz losowe słowo do zahashowania.

Następnie powinieneś obliczyć skrót SHA-512 otrzymanego słowa, kodując wynik w formacie szesnastkowym (hex).

Po obliczeniu hash'a wyślij go do serwera metodą HTTP `POST` na endpoint `http://127.0.0.1:10003/submit`, przekazując w formacie JSON zarówno identyfikator sesji, jak i obliczony skrót SHA-512 (hex). Serwer zweryfikuje poprawność przesłanego hash'a i zwróci informację o sukcesie lub błędzie.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 10003:10003 mazurkatarzyna/hashing-sha-512-ex1:latest
```

(b) Do obliczenia skrótu SHA-512 użyj narzędzia `OpenSSL`.

(c) Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dołączeniu nagłówka HTTP `"Content-Type: application/json"`.

```

[user@parrot]-[~]
→ $curl -X GET http://localhost:10003/hash
{"session_id":"16cf86bf419b6e9b","word":"segnad"}
[user@parrot]-[~]
→ $echo -n "segnad" | openssl dgst -SHA-512
SHA2-512(stdin)= 96792c55851e5ed035db03ad0b919c9ca71897549554356e4cb7dbe0ed3233de4508d3a7a0f70b5d6917f3e9faf70958b877287394747931d19b947293c81caa
[user@parrot]-[~]
→ $curl -X 'POST' \
'http://localhost:10003/submit' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "16cf86bf419b6e9b",
  "hash_hex": "96792c55851e5ed035db03ad0b919c9ca71897549554356e4cb7dbe0ed3233de4508d3a7a0f70b5d6917f3e9faf70958b877287394747931d19b947293c81caa"
}'
{"result":"Correct SHA-512 hash!"}

```

- 1.4 Pod adresem <http://127.0.0.1:10004> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:10004/hash> oraz <http://127.0.0.1:10004/submit>.

Twoim zadaniem jest obliczenie skrótu **Argon** podanego przez serwer losowego słowa. Wynikowy hash powinien być w formacie szesnastkowym (hex). Aby otrzymać od serwera słowo do zahashowania, parametry użyte do haszowania oraz identyfikator sesji, wyślij zapytanie HTTP *GET* do endpointa <http://127.0.0.1:10004/hash>. Otrzymasz w odpowiedzi unikalny identyfikator sesji, losowe słowo do zahashowania i niezbędne parametry.

Następnie powinieneś obliczyć skrót **Argon** otrzymanego słowa, kodując wynik w formacie szesnastkowym (hex).

Po obliczeniu hash'a wyślij go do serwera metodą HTTP *POST* na endpoint <http://127.0.0.1:10004/submit>, przekazując w formacie JSON zarówno identyfikator sesji, jak i obliczony skrót **Argon** (hex). Serwer zweryfikuje poprawność przesłanego hash'a i zwróci informację o sukcesie lub błędzie.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 10004:10004 mazurkatarzyna/hashing-argon-ex1:latest
```

- (b) Do obliczenia skrótu Argon narzędzia OpenSSL dostępnego jako kontener Dockerowy. W tym celu uruchom kontener, i wejdź do jego środka. Haszowanie za pomocą algorytmu Argon dostępne jest pod nazwą `kdf`.

```
docker run -it mazurkatarzyna/openssl-332-ubuntu:latest
openssl kdf -help
```

- (c) Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dołączeniu nagłówka HTTP `"Content-Type: application/json"`.

```
[user@parrot]~  
$ curl -X GET http://localhost:10004/hash  
{  
  "kdf_options": {  
    "iter": 1,  
    "keylen": 24,  
    "memcost": 8192,  
    "salt": "NaCl2024"  
  },  
  "session_id": "8a9ba0f1bc8a2d38",  
  "word": "nieslawienska"  
}
```

```
[user@parrot]~  
$ podman run -it docker.io/mazurkatarzyna/openssl-332-ubuntu:latest  
Trying to pull docker.io/mazurkatarzyna/openssl-332-ubuntu:latest...  
Getting image source signatures  
Copying blob f24d33ed7339 done  
Copying blob 7d4bbb6a9eae done  
Copying blob 1d387567261e done  
Copying blob 3f34f8369f71 done  
Copying blob f186222ac064 done  
Copying blob 038e7ff2d571 done  
Copying blob 965acd2b4f69 done  
Copying blob f6d24776cde2 done  
Copying blob e610e957fc29 done  
Copying blob 03b997b92b54 done  
Copying blob b72ad92f63fc done  
Copying blob a68e8327e7dc done  
Copying blob cfc5b07973c done  
Copying blob 0ac11a9723c2 done  
Copying blob 40225ac59a78 skipped: already exists  
Copying blob 7dc94baab4d3 done  
Copying blob eebae4277690 done  
Copying blob 4f4fb700ef54 done  
Copying blob c695ceb5f249 done  
Copying blob 438765a91c7f done  
Copying config 2ab1cd8112 done  
Writing manifest to image destination  
Storing signatures  
root@6ccd7b4d68ce:/app# openssl kdf -keylen 24 -kdfopt pass:nieslawienska -kdfopt salt:NaCl2024  
-kdfopt iter:1 -kdfopt memcost:8192 ARGON2D | tr -d ':' | tr '[:upper:]' '[:lower:]'  
D7CDEC001D366145EAA2730B58EADB17D2BD61D5EC66F76E
```

```
[user@parrot]~$ curl -X 'POST' \
'http://localhost:10004/submit/argon2d' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "8a9ba0f1bc8a2d38",
  "hash_hex": "D7CDEC001D366145EAA2730B58EADB17D2BD61D5EC66F76E"
}'
{
  "result": "Correct ARGON2D hash!"
}
```

- 1.5 Pod adresem `http://127.0.0.1:10005` działa prosty serwer HTTP udostępniający dwa endpointy: `http://127.0.0.1:10005/hash` oraz `http://127.0.0.1:10005/submit`.

Twoim zadaniem jest obliczenie skrótu **bcrypt** podanego przez serwer losowego słowa. Wynikowy hash powinien być w formacie szesnastkowym (hex). Aby otrzymać od serwera słowo do zahashowania oraz identyfikator sesji, wyślij zapytanie HTTP `GET` do endpointa `http://127.0.0.1:10005/hash`. Otrzymasz w odpowiedzi unikalny identyfikator sesji i losowe słowo do zahashowania.

Następnie powinieneś obliczyć skrót **bcrypt** otrzymanego słowa, kodując wynik w formacie szesnastkowym (hex).

Po obliczeniu hash'a wyślij go do serwera metodą HTTP `POST` na endpoint `http://127.0.0.1:10005/submit`, przekazując w formacie JSON zarówno identyfikator sesji, jak i obliczony skrót **bcrypt** (hex). Serwer zweryfikuje poprawność przesłanego hash'a i zwróci informację o sukcesie lub błędzie.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 10005:10005 mazurkatarzyna/hashing-bcrypt-ex1:latest
```

- (b) Do obliczenia skrótu **bcrypt** użyj narzędzia `htpasswd` dostępnego jako kontener Dockerowy.

```
docker run mazurkatarzyna/htpasswd:latest --help
```

- (c) Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dołączeniu nagłówka HTTP `"Content-Type: application/json"`.

```
[user@parrot]~  
$ curl -X 'GET' \br/>  'http://localhost:10005/hash' \br/>  -H 'accept: application/json'  
{br/>  "session_id": "adedf1444f3f5a91",br/>  "word": "nieredukowana"  
}
```

```
[x]-[user@parrot]~  
$ podman run docker.io/mazurkatarzyna/htpasswd:latest -nbB username nieredukow  
ana  
username:$2y$05$ZB0RoanKyFERb172vQswv.2ykYBv1Te3rQeal1Q8Nrbs9hndowqEy  
  
[user@parrot]~  
$ curl -X 'POST' \br/>  'http://localhost:10005/submit' \br/>  -H 'accept: application/json' \br/>  -H 'Content-Type: application/json' \br/>  -d '{  
    "session_id": "adedf1444f3f5a91",  
    "hash_hex": "$2y$05$ZB0RoanKyFERb172vQswv.2ykYBv1Te3rQeal1Q8Nrbs9hndowqEy"  
  }'  
{  
  "result": "Correct bcrypt hash!"  
}
```

Usage:

```
htpasswd [-cimB25dpsDv] [-C cost] [-r rounds] passwordfile username  
htpasswd -b[cmB25dpsDv] [-C cost] [-r rounds] passwordfile username password
```

rd

```
htpasswd -n[imB25dps] [-C cost] [-r rounds] username
```

```
htpasswd -nb[mB25dps] [-C cost] [-r rounds] username password
```

-c Create a new file.

-n Don't update file; display results on stdout.

-b Use the password from the command line rather than prompting for it.

-i Read password from stdin without verification (for script usage).

-m Force MD5 hashing of the password (default).

-2 Force SHA-256 hashing of the password (secure).

-5 Force SHA-512 hashing of the password (secure).

-B Force bcrypt hashing of the password (very secure).

-C Set the computing time used for the bcrypt algorithm
(higher is more secure but slower, default: 5, valid: 4 to 17).

-r Set the number of rounds used for the SHA-256, SHA-512 algorithms
(higher is more secure but slower, default: 5000).

-d Force CRYPT hashing of the password (8 chars max, insecure).

-s Force SHA-1 hashing of the password (insecure).

-p Do not hash the password (plaintext, insecure).

-D Delete the specified user.

-v Verify password for the specified user.

1.6 Napisz skrypt w języku Python, w którym wygenerujesz hash **MD5** dowolnego ciągu znaków podawanego jako argument wywołania skryptu. Sprawdź poprawność wygenerowanego hasha porównując go z wynikiem otrzymanym przy pomocy `md5sum` lub `openssl`. Wykorzystaj bibliotekę [hashlib](#). Możesz wykorzystać poniższy szkielet kodu:

```
import hashlib

def md5_hash_string(input_string: str) -> str:
    """
    Returns the MD5 hash of a given string.
    """
    pass

if __name__ == "__main__":
    text = "hello world"
    print(f"MD5 hash of '{text}': {md5_hash_string(text)}")
```

```
[user@parrot]~$ touch zad1_6.py
[user@parrot]~$ vim zad1_6.py
```

```
1 import hashlib as hs$
2 $
3 def md5_hash_string(input_string: str) -> str:$
4     hash_text=hs.md5(input_string.encode())$
5     return hash_text.hexdigest()$
6 $
7 if __name__ == "__main__":$
8     text = "hello world"$
9     print(f"MD5 hash of '{text}': {md5_hash_string(text)}")$
```

zad1_6.py

```
[user@parrot]~  
$python3 zad1_6.py  
MD5 hash of 'hello world': 5eb63bbbe01eeed093cb22bb8f5acdc3
```

- 1.7 Napisz skrypt w języku Python, w którym wygenerujesz hash **SHA-1** dowolnego ciągu znaków podawanego jako argument wywołania skryptu. Sprawdź poprawność wygenerowanego hasha porównując go z wynikiem otrzymanym przy pomocy `sha1sum` lub `openssl`. Wykorzystaj bibliotekę [hashlib](#). Możesz wykorzystać poniższy szkielet kodu:

```
import hashlib  
  
def sha1_hash_string(input_string: str) -> str:  
    """  
    Returns the SHA-1 hash of a given string.  
    """  
    pass  
  
if __name__ == "__main__":  
    text = "hello world"  
    print(f"SHA-1 hash of '{text}': {sha1_hash_string(text)}")
```

```
[user@parrot]~  
$touch zad1_7.py  
[user@parrot]~  
$vim zad1_7.py
```

```
1 import hashlib as hs$  
2 $  
3 def sha1_hash_string(input_string: str) -> str:$  
4     hash_text=hs.sha1(input_string.encode())$  
5     return hash_text.hexdigest()$  
6 $  
7 if __name__ == "__main__":$  
8     text = "hello world"$  
9     print(f"SHA-1 hash of '{text}': {sha1_hash_string(text)}")$
```

```
[user@parrot]~  
$python3 zad1_7.py  
SHA-1 hash of 'hello world': 2aae6c35c94fcfb415dbe95f408b9ce91ee846ed
```

- 1.8 Napisz skrypt w języku Python, w którym wygenerujesz hash **SHA-256** dowolnego ciągu znaków podawanego jako argument wywołania skryptu. Sprawdź poprawność wygenerowanego hasha porównując go z wynikiem otrzymanym przy pomocy `openssl`. Wykorzystaj bibliotekę [hashlib](#). Możesz wykorzystać poniższy szkielet kodu:

```
import hashlib

def sha256_hash_string(input_string: str) -> str:
    """
    Returns the SHA-256 hash of a given string.
    """
    pass

if __name__ == "__main__":
    text = "hello world"
    print(f"SHA-256 hash of '{text}': {sha256_hash_string(text)}")
```

```
[user@parrot]~$ touch zad1_8.py
[user@parrot]~$ vim zad1_8.py
```

```
1 import hashlib as hs$
2 $
3 def sha256_hash_string(input_string: str) -> str:$
4     hash_text = hs.sha256(input_string.encode())$
5     return hash_text.hexdigest()$
6 $
7 if __name__ == "__main__":$
8     text = "hello world"$
9     print(f"SHA-256 hash of '{text}': {sha256_hash_string(text)}")$
```

```
[user@parrot]~$ python3 zad1_8.py
SHA-256 hash of 'hello world': b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9
```

1.9 Napisz skrypt w języku Python, w którym wygenerujesz hash **bcrypt** dowolnego ciągu znaków podawanego jako argument wywołania skryptu. Sprawdź poprawność wygenerowanego hasha porównując go z wynikiem otrzymanym przy pomocy **htpasswd**. Do napisania programu potrzebna jest biblioteka [bcrypt](#) zainstalowana w kontenerze Dockerowym. Uruchom kontener, i wejdź do jego środka - biblioteka jest już zainstalowana w kontenerze. Aby napisać program, wykorzystaj **nano**.

(a) Uruchom kontener za pomocą poniższego polecenia:

```
docker run -it mazurkatarzyna/hashing-bcrypt-ex2:latest
nano template.py
```

(b) Do weryfikacji skrótu **bcrypt** użyj narzędzia **htpasswd** dostępnego jako kontener Dockerowy.

```
docker run mazurkatarzyna/htpasswd:latest --help
```

Możesz wykorzystać poniższy szkielet kodu, który dostępny jest również wewnątrz kontenera jako plik `template.py`:

```
import bcrypt

def bcrypt_hash_password(password: str) -> str:
    """
    Hash a password using bcrypt.
    """
    pass

def bcrypt_verify_password(password: str, hashed: str) -> bool:
    """
    Verify a password against a bcrypt hash.
    """
    pass

# Example usage
if __name__ == "__main__":
    pwd = "my_secure_password"
```

```
[user@parrot]-[~]
$podman run -it docker.io/mazurkatarzyna/hashing-bcrypt-ex2:latest
Trying to pull docker.io/mazurkatarzyna/hashing-bcrypt-ex2:latest...
Getting image source signatures
Copying blob b7b61708209a done
Copying blob 59e22667830b done
Copying blob 657eaaa70199 done
Copying blob 4085babbc570 done
Copying blob 5b3bd8c36be4 done
Copying blob abd846fa1cdb done
Copying blob 29640ba723f3 done
Copying blob 481ffe8ee85a done
Copying config a4e6e7d42b done
Writing manifest to image destination
Storing signatures
root@51f5aba7d55c:/app# ls
template.py
```

```
1  import bcrypt
2
3  def bcrypt_hash_password(password: str) -> str:
4      hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
5      return hashed.decode()
6
7  def bcrypt_verify_password(password: str, hashed: str) -> bool:
8      return bcrypt.checkpw(password.encode(), hashed.encode())
9
10 # Example usage
11 if __name__ == "__main__":
12     pwd = "my_secure_password"
13     hashed_pwd=bcrypt_hash_password(pwd)
14     print(f"Oryginalne haslo: {pwd}")
15     print(f"Wygenerowany hash bcrypt: {hashed_pwd}")
16
17     if bcrypt_verify_password(pwd, hashed_pwd):
18         print("Poprawne haslo")
19     else:
20         print("Niepoprawne haslo")
```

```
root@51f5aba7d55c:/app# python3 template.py
Oryginalne haslo: my_secure_password
Wygenerowany hash bcrypt: $2b$12$opn07oi8hwercinuQK9RcuFSy1N3fKXdrP7PzcmVcNn7cWSWMh1oe
Poprawne haslo
```



```

1 def porownaj_stringi(str1, str2):
2     if str1 == str2:
3         print("Stringi są identyczne.")
4     else:
5         print("Stringi są różne.")
6         # pokaż różnice znak po znaku
7         for i in range(min(len(str1), len(str2))):
8             if str1[i] != str2[i]:
9                 print(f"Różnica na pozycji {i}: '{str1[i]}' vs '{str2[i]}'")
10        # sprawdź, czy jeden string jest dłuższy
11        if len(str1) != len(str2):
12            print("Dodatkowe znaki:")
13            if len(str1) > len(str2):
14                print(f"'{str1[len(str2):]}' w pierwszym stringu")
15            else:
16                print(f"'{str2[len(str1):]}' w drugim stringu")
17
18 # przykładowe użycie
19 porownaj_stringi("6adfb183a4a2c94a2f92dab5ade762a47889a5a1",
20                  "6adfb183a4a2c94a2f92dab5ade762a47889a5a1")

```

```

[user@parrot]~$
$python3 template.py
Stringi są identyczne.

```

- 1.11 Mając dany początkowy ciąg znaków `helloworld`, który następnie został zahashowany, określ, jaka funkcja skrótu została wykorzystana do utworzenia hashu: `$2y$10$xyAv5a46CQYPay5UISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa`. Możesz wykorzystać narzędzie `hash-identifier`:

```
docker run -it mazurkatarzyna/hash-identifier:latest
```

Czy `hash-identifier` odnalazł nazwę hashu? Jeśli nie, zaproponuj inny sposób na rozwiązanie zadania.

```

[user@parrot]~]
$podman run -it docker.io/mazurkatarzyna/hash-identifier:latest
#####
#                                     #
#  /\  /\  /\                        /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#  /\  /\  /\  /\                    /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  /\  #
#                                     #
#                                     By Zion3R #
#                                     www.Blackploit.com #
#                                     Root@Blackploit.com #
#                                     #
#####
-----
HASH: $2y$10$xbyAv5a46CQYPay5UIISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa

Not Found.

```

```

[user@parrot]~]
$hashid
$2y$10$xbyAv5a46CQYPay5UIISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa
Analyzing '$2y$10$xbyAv5a46CQYPay5UIISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa'
[+] Blowfish(OpenBSD)
[+] Woltlab Burning Board 4.x
[+] bcrypt

```

```

[user@parrot]~]
$openssl list -cipher-algorithms | grep "blowfish"
blowfish => BF-CBC

```

Sprawdzenie - hashcat

```

[user@parrot]~]
$nano hash.txt

```

```

GNU nano 7.2 hash.txt
$2y$10$xbyAv5a46CQYPay5UIISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa

```

```

[user@parrot]~]
$echo -n "helloworld" > word.txt

```

hash-mode 3200 bo:

3200	bcrypt 2 *, Blowfish (Unix)	\$2a\$05\$LhayLxezLhK1LhWvKxCyLOj0j1u.Kj0jZ0pEmm134uzrQlF
------	-----------------------------	---

```
[user@parrot]~  
$hashcat -m 3200 -a 0 hash.txt word.txt
```

```
[user@parrot]~  
$hashcat -m 3200 hash.txt --show  
$2y$10$xbyAv5a46CQYPay5UISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa:hellworld
```

- 1.12 Mając dany początkowy ciąg znaków **hellworld**, który następnie został zahashowany, określ, jaka funkcja skrótu została wykorzystana do utworzenia hasha: **6adfb183a4a2c94a2f92dab5ade762a47889a5a1**. Możesz wykorzystać narzędzie **hashid**:

```
docker run mazurkatarzyna/hashid:latest yourhash
```

Czy **hashid** odnalazł nazwę hasha? Jeśli nie, zaproponuj inny sposób na rozwiązanie zadania.

```
[user@parrot]~  
$podman run docker.io/mazurkatarzyna/hashid:latest 6adfb183a4a2c94a2f92dab5ade762a47889a5a1  
Trying to pull docker.io/mazurkatarzyna/hashid:latest...  
Getting image source signatures  
Copying blob 59e22667830b skipped: already exists  
Copying blob 4c665aba06d1 done  
Copying blob e3586b415667 done  
Copying blob f5cc5422ebcb done  
Copying blob 1f7fc2b4885c done  
Copying config b83df94cf0 done  
Writing manifest to image destination  
Storing signatures  
Analyzing '6adfb183a4a2c94a2f92dab5ade762a47889a5a1'  
[+] SHA-1  
[+] Double SHA-1  
[+] RIPEMD-160  
[+] Haval-160  
[+] Tiger-160  
[+] HAS-160  
[+] LinkedIn  
[+] Skein-256(160)  
[+] Skein-512(160)  
[user@parrot]~  
$echo -n "hellworld" | openssl dgst -sha-1  
SHA1(stdin)= 6adfb183a4a2c94a2f92dab5ade762a47889a5a1
```

- 1.13 Mając dany początkowy ciąg znaków `helloworld`, który następnie został zahashowany, określ, jaka funkcja skrótu została wykorzystana do utworzenia hasha: `$2y$10$xbyAv5a46CQYPay5UISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa`. Możesz wykorzystać narzędzie `hashid`:

```
docker run mazurkatarzyna/hashid:latest yourhash
```

Czy `hashid` odnalazł nazwę hasha? Jeśli nie, zaproponuj inny sposób na rozwiązanie zadania.

```
[user@parrot]-[~]
└─ $hashid
$2y$10$xbyAv5a46CQYPay5UISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa
Analyzing '$2y$10$xbyAv5a46CQYPay5UISCNeFWpVdx2qvhCBE0Z/YtfxoVXh0GrVKQa'
[+] Blowfish(OpenBSD)
[+] Woltlab Burning Board 4.x
[+] bcrypt
```

-
- 1.14 Mając dany początkowy ciąg znaków `R3iSrSnmG9SFHxVekUD`, który następnie został zahashowany, określ, jaka funkcja skrótu została wykorzystana do utworzenia hasha: `48cab4b54bef42fddaa6353c68a20b369f40026e`. Aby rozwiązać zadanie, napisz skrypt w języku Python. Podpowiedź: algorytm jest dostępny w bibliotece `hashlib`.

```

import hashlib

# Przykładowe dane
input_text = "R3iSrSNmg9SFHxVakUD"
target_hash = "48cab4b54bef42fddaa6335c68a20b369f4002d"

def find_matching_algorithms(text: str, target: str):
    matches = []

    # używamy algorithms_available - daje szerszy zestaw algorytmów dostępnych na
    systemie
    for algo in sorted(hashlib.algorithms_available):
        try:
            # Hashujemy tekst przy pomocy algorytmu
            h = hashlib.new(algo, text.encode()).hexdigest()
        except (ValueError, TypeError):
            # Nieobsługiwany algorytm w tej konfiguracji Python
            continue

        # Porównujemy obliczony hash z podanym hashem
        if h == target.lower():
            matches.append((algo, h))

    return matches

if __name__ == "__main__":
    # Wywołanie funkcji
    res = find_matching_algorithms(input_text, target_hash)

    if res:
        print("Znaleziono dopasowanie:")
        for algo, h in res:
            print(f"Algorytm: {algo} → {h}")
    else:
        print("Brak dopasowania w hashlib.algorithms_available.")
        print("Upewnij się, że używasz pełnej listy algorytmów
(algorithms_available) lub zainstaluj dodatkowe algorytmy (np. RIPEMD-160) w Twojej
wersji OpenSSL/Python.")

```

```

Znaleziono dopasowanie:
- algorytm: ripemd160 -> 48cab4b54bef42fddaa6353c68a20b369f40026e

```